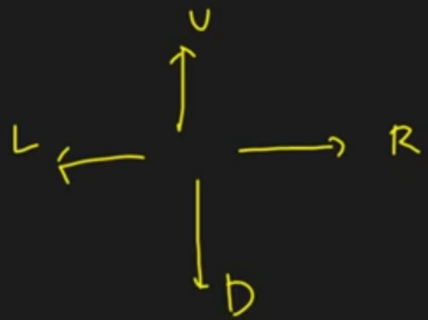
	0	1	2	3
0	1	0	0	0
1	1	1	0	1
2	1	1	0	0
3	0	1	1	1



DDRDRR < DRDDRR

L19. Rat in A Maze | Backtracking

	0	1	2	3
0	1	0	0	0
1	1	1	0	1
2	1	1	0	0
3	0	1	1	1

✓			
✓			
✓	✓		
	✓	✓	

vis

f(0, 0, "") (D|L|R|U)

f(1, 0, "D") D|L|R|U

f(2, 0, "DD") ~~D~~|~~L~~|R|U

f(2, 1, DDR) D|L|R|U

f(3, 1, DDRD) D|L|R|U

f(3, 2, DDRDR) D|L|R|U

f(3, 3) DDRDRR

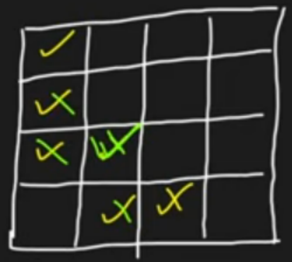
D L R U

DDRDRR

10:09 / 25:09

L19. Rat in A Maze | Backtracking

	0	1	2	3
0	1	0	0	0
1	1	1	0	1
2	1	1	0	0
3	0	1	1	1



vis

f(0,0,"") (D|L|R|U)

f(1,0,"D") D|L|R|U

f(2,0,"DD") ~~D~~|~~L~~|R|U

f(2,1,DDR) D|~~L~~|~~R~~|U

f(1,1,DDRU) ~~D~~|~~L~~|~~R~~|~~U~~

f(3,1,DDRD) D/L/R/U

f(3,2,DDRDR) D/L/R/U

f(3,3) DDRDRR

D L R U

DDRDRR

13:08 / 25:09

L19. Rat in A Maze | Backtracking

2.20

	0	1	2	3
0	1	0	0	0
1	1	1	0	1
2	1	1	0	0
3	0	1	1	1

vis

f(0, 0, "") (D | L | R | U)

D L R U

f(1, 0, "D") D | L | R | U

→ DDRDRR

→ DRDDRR

f(2, 0, "DD") D | L | R | U

f(1, 1, DR) D | L | R | U

f(2, 1, DDR) D | L | R | U

f(2, 1, DRD) D | L | R | U

f(1, 1, DDRU) D | L | R | U

f(3, 1, DDRD) D | L | R | U

f(3, 1, DRDD) D | L | R | U

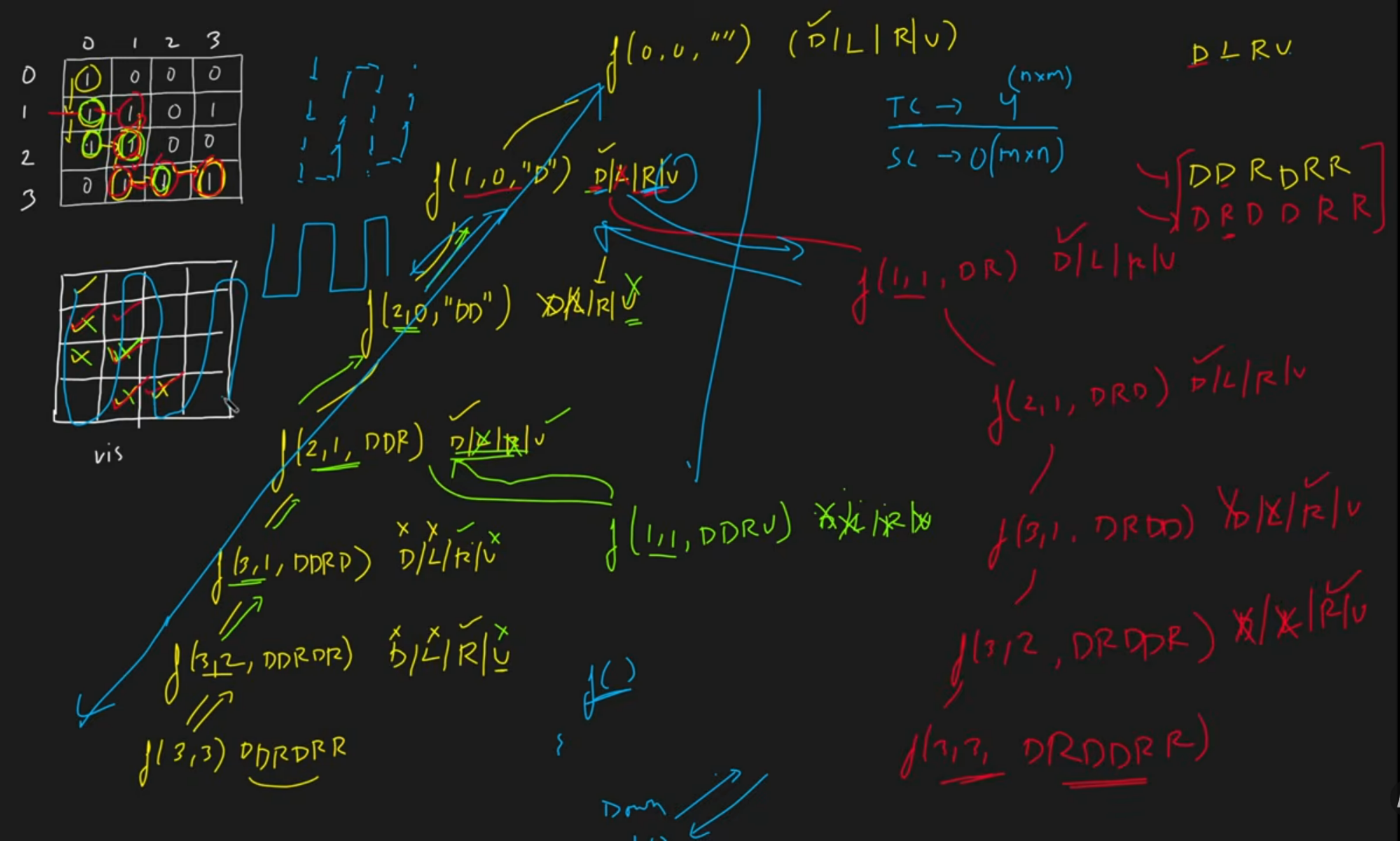
f(3, 2, DDRDR) D | L | R | U

f(3, 2, DRDDR) D | L | R | U

f(3, 3) DDRDRR

f(3, 3, DRDDRR)

15:54 / 25:09



0 1 2 3
vis
f(0, 0, "") D/L/R/U
f(1, 0, "D") D/L/R/U
f(2, 0, "DD") D/L/R/U
f(2, 1, DDR) D/L/R/U
f(1, 1, DDRU) D/L/R/U
f(3, 1, DDRD) D/L/R/U
f(3, 2, DDRDR) D/L/R/U
f(3, 3) DDRDRR
f(1, 1, DR) D/L/R/U
f(2, 1, DRD) D/L/R/U
f(3, 1, DRDD) D/L/R/U
f(3, 2, DRDDR) D/L/R/U
f(3, 3, DRDDRR)
D L R U
TC → 4^(n×m)
SC → O(m×n)
DDRDRR
DRDDRR

vis

f(2,1, DDR) D/~~L~~/~~R~~/U

f(1,1, DDRU) ~~D~~/~~L~~/~~R~~/~~U~~

f(3,1, DDRD) D/L/R/U

f(3,2, DDRDR) D/L/R/U

f(3,3) DDRDRR

f(2,1, DRD) D/L/R/U

f(3,1, DRDD) ~~D~~/~~L~~/R/U

f(3,2, DRDDR) ~~D~~/~~L~~/R/U

f(3,3, DRDDRR)

f()

{

Down f()

Left f()

R f()

U f()

}

L19. Rat in A Maze | Backtracking

```cpp
// User function template for C++

class Solution{
    void solve(int i, int j, vector<vector<int>> &a, int n, vector<string> &ans, string move, vector<vector<int>> &vis) {
        if(i==n-1 && j==n-1) {
            ans.push_back(move);
            return;
        }
        // downward
        if(i+1<n && !vis[i+1][j] && a[i+1][j] == 1) {
            vis[i][j] = 1;
            solve(i+1, j, a, n, ans, move + 'D', vis);
            vis[i][j] = 0;
        }

        // left
        if(j-1>=0 && !vis[i][j-1] && a[i][j-1] == 1) {
            vis[i][j] = 1;
            solve(i, j-1, a, n, ans, move + 'L', vis);
            vis[i][j] = 0;
        }

        // right
        if(j+1<n && !vis[i][j+1] && a[i][j+1] == 1) {
            vis[i][j] = 1;
            solve(i, j+1, a, n, ans, move + 'R', vis);
            vis[i][j] = 0;
        }

        // upward
        if(i-1>=0 && !vis[i-1][j] && a[i-1][j] == 1) {
            vis[i][j] = 1;
            solve(i-1, j, a, n, ans, move + 'U', vis);
            vis[i][j] = 0;
        }
    }
    public:
    vector<string> findPath(vector<vector<int>> &m, int n) {
        vector<string> ans;
```

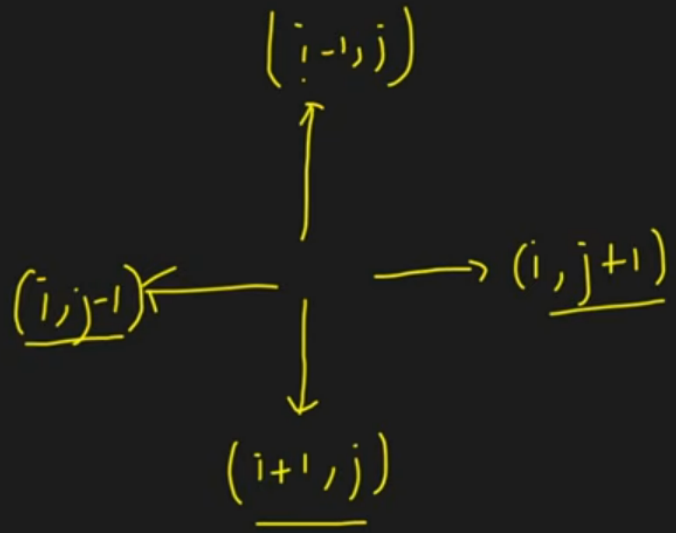
20:55 / 25:09

```
            return;
        }
        // downward
        if(i+1<n && !vis[i+1][j] && a[i+1][j] == 1) {
            vis[i][j] = 1;
            solve(i+1, j, a, n, ans, move + 'D', vis);
            vis[i][j] = 0;
        }

        // left
        if(j-1>=0 && !vis[i][j-1] && a[i][j-1] == 1) {
            vis[i][j] = 1;
            solve(i, j-1, a, n, ans, move + 'L', vis);
            vis[i][j] = 0;
        }

        // right
        if(j+1<n && !vis[i][j+1] && a[i][j+1] == 1) {
            vis[i][j] = 1;
            solve(i, j+1, a, n, ans, move + 'R', vis);
            vis[i][j] = 0;
        }

        // upward
        if(i-1>=0 && !vis[i-1][j] && a[i-1][j] == 1) {
            vis[i][j] = 1;
            solve(i-1, j, a, n, ans, move + 'U', vis);
            vis[i][j] = 0;
        }
    }
    public:
    vector<string> findPath(vector<vector<int>> &m, int n) {
        vector<string> ans;
        vector<vector<int>> vis(n, vector<int> (n, 0));
        if(m[0][0] == 1) solve(0,0,m,n, ans, "", vis);
        return ans;
    }
};
```



D | L | R | U

	D	L	R	U
di[] =	+1	+0	+0	-1
dj[] =	+0	-1	+1	+0

$\Rightarrow i + di[ind]$

$\Rightarrow j + dj[ind]$

```cpp

class Solution{
    void solve(int i, int j, vector<vector<int>> &a, int n, vector<string> &ans, string move,
    vector<vector<int>> &vis, int di[], int dj[]) {
        if(i==n-1 && j==n-1) {
            ans.push_back(move);
            return;
        }
        string dir = "DLRU";
        for(int ind = 0; ind<4;ind++) {
            int nexti = i + di[ind];
            int nextj = j + dj[ind];
            if(nexti >= 0 && nextj >= 0 && nexti < n && nextj < n && !vis[nexti][nextj] && a[nexti][nextj] == 1) {
                vis[i][j] = 1;
                solve(nexti, nextj, a, n, ans, move + dir[ind], vis, di, dj);
                vis[i][j] = 0;
            }
        }
    }
    public:
    vector<string> findPath(vector<vector<int>> &m, int n) {
        vector<string> ans;
        vector<vector<int>> vis(n, vector<int> (n, 0));
        int di[] = {+1, 0, 0, -1};
        int dj[] = {0, -1, 1, 0};
        if(m[0][0] == 1) solve(0,0,m,n, ans, "", vis, di, dj);
        return ans;
    }
};



} // } Driver Code Ends
```